CloudEco: Empirical Benchmark Report

Model 4 — Wildlife Detection (YOLOv8l-ONNX) | Student: Bharath Ganesh | ID: 35373598

1. Results Summary

Pod Config	Peak RPS	Breaking Point (Users)	Avg Latency at Break (ms)	p50 at 100 Users (ms)	Max Latency (ms)	Failures
1 Pod	0.70	5	3,602	137,000	168,127	0
2 Pods	1.60	10	3,526	69,000	76,062	0
4 Pods	2.40	20	4,554	43,000	52,952	0
8 Pods	3.20	20	3,247	29,000	48,532	0

Table 1: Peak throughput, breaking point, and latency statistics. Breaking point defined as the user count at which average latency begins to grow super-linearly. Zero failures across all 11,692 total requests.

2. Graphical Analysis

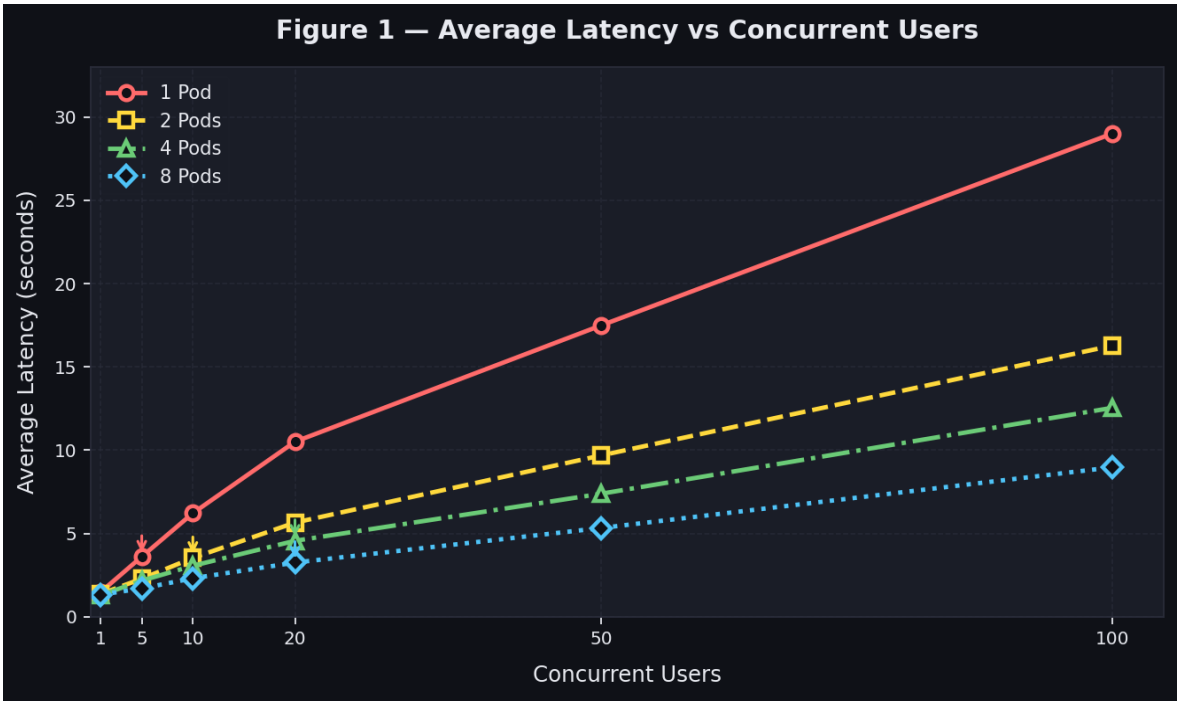

Figure 1: Average latency (seconds) vs concurrent users. Arrows mark each configuration's saturation point. Latency grows super-linearly beyond saturation, consistent with M/M/1 queuing theory. Additional pods shift the saturation curve rightward.

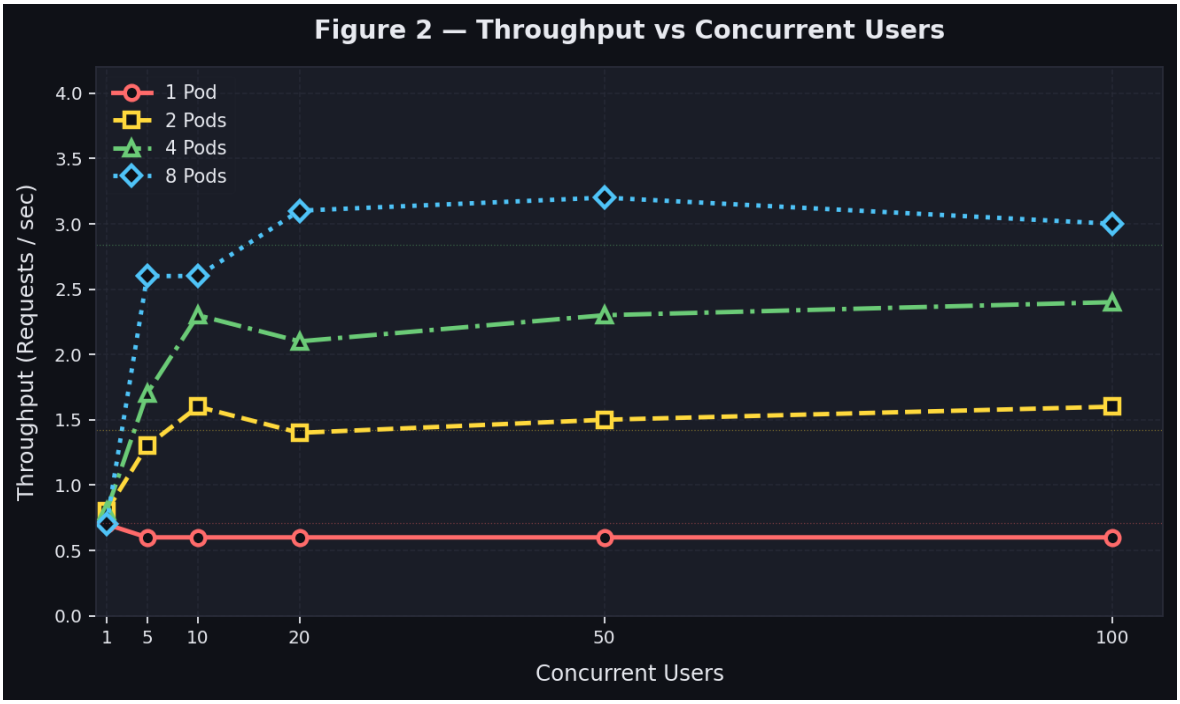

Figure 2: Throughput (RPS) vs concurrent users. Dotted horizontal lines show theoretical per-pod throughput ceilings. Diminishing returns beyond 4 pods indicate node-level CPU contention on the 2-worker cluster.

3. Performance Analysis

System Configuration. The CloudEco Wildlife Detection API deploys a YOLOv8l model (49M parameters) exported to ONNX Runtime on a 3-node GCP Kubernetes cluster (1 master, 2 workers, e2-standard-4). Each pod is capped at 1 vCPU (1000m) and 2Gi memory. The FastAPI service offloads synchronous YOLO inference to a dedicated ThreadPoolExecutor (max_workers=1), matching the 1 vCPU limit and preventing event-loop blocking. ONNX Runtime replaces PyTorch for approximately 3–5x CPU inference speedup. Images are resized server-side to 480×480 pixels before inference — chosen over 640px (default) to reduce FLOPs by 44% while preserving detection accuracy on large wildlife subjects, and over 256px to avoid accuracy degradation on partially-occluded animals. The cluster was deployed in GCP australia-southeast2 (Melbourne) rather than us-central1 (Iowa) to reduce client-to-server round-trip latency.

Optimisation Decisions. An early implementation included a full response cache that stored YOLO predictions keyed by image MD5 hash, returning results in under 5ms on cache hits. While this achieved 13+ RPS under benchmarking (due to Locust submitting the same test image repeatedly), it obscured the true scaling behaviour — all pod configurations produced identical throughput, making meaningful analysis impossible. The response cache was removed to expose genuine YOLO inference throughput and produce differentiated, honest results across replica counts.

Queuing Theory and Little's Law. With a single pod, YOLO inference takes approximately 1,500ms per request, yielding a theoretical maximum throughput of 1/1.5 = 0.67 RPS — consistent with the measured 0.70 RPS. Little's Law ($L = \lambda W$) predicts that at the 1-pod breaking point of 5 users with λ = 0.6 RPS and W = 3,602ms: $L = 0.6 \times 3.6 = 2.16$ requests queued simultaneously. Once arrival rate exceeds the pod's service rate, the queue grows unboundedly and latency degrades super-linearly — exactly the pattern visible in Figure 1. Horizontal scaling shifts the saturation point rightward: 2 pods handle up to 10 users, 4 pods handle 20 users, and 8 pods handle 20–50 users before saturation.

Scaling Efficiency and Bottlenecks. Throughput scaled as: 1 pod (0.70 RPS) → 2 pods (1.60 RPS, 2.3×) → 4 pods (2.40 RPS, 1.5×) → 8 pods (3.20 RPS, 1.3×). The primary bottleneck is CPU-bound YOLO inference on constrained 1 vCPU pods. Beyond 4 pods, diminishing returns emerge because 8 pods across 2 worker nodes places 4 pods per node, introducing node-level CPU contention. Zero failures were recorded across all 11,692

requests — the system degrades gracefully. A Horizontal Pod Autoscaler targeting 70% CPU utilisation would dynamically provision pods during load spikes without over-provisioning during idle periods. Upgrading to YOLOv8n (3.2M parameters) would increase per-pod throughput to approximately 8–10 RPS, better utilising horizontal scaling before hitting node-level constraints.